VOICE VERIFICATION WITH NOISE

Liam Degand **Leland Sion** **Lilly Ko** **Yilun Shi**
Music Information Retrieval, University of Victoria, Canada
{liamdegand, lelands, lillyko, yilunshi}@uvic.ca

1. INTRODUCTION

Voice-based authentication has emerged as a convenient and secure method for device access control. However, real-world deployment scenarios often involve challenging acoustic conditions, including background music and environmental noise, which can significantly degrade verification performance [1]. This paper presents a noise-robust voice verification system designed for device-unlock authentication under signal-to-noise ratios (SNRs) as low as 0 dB. The system employs text-dependent speaker verification using random word prompts to authenticate enrolled users while reducing vulnerability to casual impostor attempts [2–4]. Our target performance criteria are greater than 90% genuine acceptance rate with less than 5% impostor acceptance rate at 0 dB SNR. This project culminates in a live demonstration system that uses real-time microphone input to detect speech, extract voice features, and authenticate users.

2. TOOLS AND DATASETS

This project implements a noise-robust, text-dependent authentication system using a Python-based audio processing pipeline. Audio is captured in real time from a microphone and processed using standard digital signal processing techniques, including framing, windowing, and spectral analysis. Mel-Frequency Cepstral Coefficients (MFCCs) are used as the primary acoustic feature because they remain widely used and effective in speaker verification and voice-based unlocking systems [2, 5]. We also employ feature normalization and aggregation to support short-utterance authentication, which is necessary for practical device-unlock scenarios [6].

To ensure reliable speech segmentation in noisy acoustic environments, a Voice Activity Detection (VAD) module is applied prior to feature extraction. Robust VAD can improve performance under diverse background noise and low-SNR conditions [7, 8]. A Gaussian Mixture Model Universal Background Model (GMM-UBM) style baseline is used as an interpretable classical reference point for speaker verification [6, 9]. In parallel, neural embedding-based methods are evaluated for improved robustness, motivated by recent work in noise-robust speech and speaker representation learning [10–12].

For training and evaluation, this project uses a combination of clean speech datasets and noise corpora to simulate realistic deployment conditions. Clean speech samples are drawn from the Mozilla Common Voice dataset, which provides diverse speakers, accents, and recording conditions under an open license [13]. To model environmental interference, background noise and music are added using the MUSAN corpus, enabling controlled mixing at SNR levels down to 0 dB [14]. In addition, robustness testing references the VOiCES corpus, which captures reverberation and real-room noise effects representative of far-field authentication scenarios [15]. Together, these tools and datasets support a reproducible and realistic evaluation of voice-based device authentication under adverse acoustic conditions.

3. RELATED WORK

Voice-based authentication has been widely studied across both continuous and text-dependent speaker verification paradigms. Early speaker recognition systems relied primarily on statistical generative models such as Gaussian Mixture Models (GMMs) to model speaker-specific acoustic characteristics from speech signals [9]. These approaches formed the foundation of many classical speaker verification systems. Later research improved the robustness of GMM-based systems by introducing quality-aware scoring methods that account for factors such as utterance duration and noise contamination, enabling more reliable verification decisions when only short speech segments are available [6]. Active and continuous voice authentication approaches analyze multiple speech segments over time instead of relying on a single phrase, which improves resilience against impersonation attempts [3]. Additionally, liveness detection techniques such as phoneme localization using stereo microphones have been proposed to mitigate replay attacks in smartphone voice authentication systems [16].

Noise-robust speech and speaker recognition remain central challenges for practical deployment. Surveys of automatic speech recognition highlight the importance of modeling background music, environmental noise, and reverberation through front-end processing and feature normalization [1]. More recent work has focused on deep learning and embedding-based speaker verification methods that learn discriminative speaker representations directly from audio. The Generalized End-to-End (GE2E)

framework proposed by Wan et al. learns a speaker embedding space where utterances from the same speaker cluster together, improving training efficiency and verification performance [11]. Architectures that jointly perform speech enhancement and speaker embedding extraction, such as extended U-Net models, have also demonstrated strong performance in noisy conditions [12]. Public datasets such as Mozilla Common Voice provide large-scale multilingual speech recordings for training and evaluation [13], while MUSAN and VOiCES support robustness testing under controlled noise and realistic far-field conditions [14, 15]. Building on this prior work, our system focuses on text-dependent voice authentication with random prompts and noise-aware preprocessing to improve robustness in real-world acoustic conditions.

4. INITIAL RESULTS

Our initial results show that the project has established a foundation for a noise-robust voice verification system. The current pipeline can capture standardized 16 kHz mono microphone audio, log recording metadata, apply normalization and preprocessing, and extract MFCC, start-frequency, FFT, and related features for downstream modelling. On the Mozilla Common Voice en-AU dataset, 3,951 usable clips from 418 speakers were prepared for analysis. Feature analysis on a filtered subset of 3,390 utterances from 152 speakers further showed clear speaker-related structure, with same-speaker and different-speaker cosine similarities of 0.6802 ± 0.2254 and 0.0446 ± 0.2753, respectively. Using these clean-data features, an initial logistic-regression baseline achieved approximately 91.8% mean validation accuracy and 98.8% top-5 accuracy.

5. STATUS REPORTS

5.1 Liam

My responsibilities focus on implementing the microphone input pipeline, improving noise robustness, and developing the verification workflow for the voice authentication system. For the audio interface, I implemented a reusable recording model in `audio.py` centered on the `MicrophoneRecorder` class. The captured audio is standardized to 16 kHz mono WAV to ensure compatibility with the MFCC processing pipeline. The start/stop recording controls are implemented in the prototype notebook interface `voice_input.ipynb`. To provide real-time recording quality feedback, I added Root Mean Square (RMS)-based monitoring in the interface. Recording metadata, including timestamp, duration, sample rate, and file path, is logged in the data folder and stored in `recordings_metadata.csv`. In addition, preprocessing such as normalization and noise gating is implemented in `audio.py`.

To improve robustness under noisy conditions, I implemented preprocessing and speech-filtering components. Noise reduction and normalization for start-frequency features are handled in `feature_startfreq.py`, while voice activity detection (VAD) is implemented in `vad.py`. These modules are integrated into the main pipeline in `pipeline.py`, with normalization imported from `normalize.py`. For verification, I implemented scoring logic as well as batch logging of predictions and scores in `pipeline.py`. After reviewing related work, particularly the Generalized End-to-End (GE2E) loss framework for speaker verification proposed by Wan et al. [11], we determined that an embedding-based approach to speaker verification is well suited to our system design. This approach improves verification performance and training efficiency by directly optimizing speaker similarity rather than relying only on classification losses.

In conclusion, I completed PI1 and PI2 for the microphone input system by capturing audio from the microphone in a standardized format and implementing a user interface for starting and stopping recordings. I also completed PI3 and PI4 by adding real-time RMS-based monitoring for recording quality and by storing recordings with metadata in the data folder. In addition, I completed PI5 for the microphone input system by implementing automated preprocessing of recorded audio through normalization and noise gating in the pipeline. For the noise-robustness component, I completed PI1 and PI2 by introducing noise-aware preprocessing and implementing basic noise isolation through filtering, normalization, and VAD. The remaining indicators for the noise-robustness component (PI3, PI4, and PI5) are marked as future work, as are the remaining indicators in the verification pipeline with random prompts.

5.2 Lilly

In this project, I built the audio normalization pipeline and connected it to the model training loop. The objective of normalization is to put raw audio into a consistent, well-conditioned format before it reaches the model. For sample rate, I used polyphase resampling to bring everything to 16,000 Hz, which is standard for speech tasks. For amplitude, I started with simple peak normalization but realized it was not sufficient. A quiet recording with one loud spike would normalize very differently from a consistently loud one. Because of this, I switched to RMS normalization targeting -20 dBFS, which scales based on the average energy of the whole clip rather than the loudest point. Doing this completed basic objective PI1, which was to standardize audio amplitude and sample rate. The other basic objective, PI2, was dynamic range compression. Without this, some recordings still had a lot of variation between quiet and loud moments. For instance, a speaker might be quieter at the start of a sentence and louder mid-word, making the MFCC features unstable across frames. Compression smooths this out by attenuating anything above a threshold and applying makeup gain to restore overall loudness. The compressor in this project works like a basic studio limiter: samples above a threshold of 0.3 are attenuated by a ratio of 4:1, and then a $1.2\times$ makeup gain is applied. There is also hard clipping to the range $[-1, 1]$.

Expected objective PI3 was to validate normaliza-

tion by comparing feature distributions before and after processing. To do this, I added a function (`compare_distributions`) that computes per-coefficient statistics before and after processing. Results are shown in Figures 1 and 2 in the appendix.

The next step was to evaluate the impact of normalization on model input consistency and training stability (expected objective PI4). Completion of this objective is shown in Figures 3 and 4 in the appendix.

The next task was to write the code to train the model to classify whether two audio samples belong to the same speaker, including forward pass, loss computation, and backpropagation. My first attempt at the training loop used a simple MLP written in NumPy. While this approach ran, it operated on averaged MFCC frames, which does not account for information on how someone's voice changes over time. To capture this detail, I moved to an LSTM-based approach with GE2E loss [11]. In this approach, the LSTM reads the full MFCC sequence frame by frame rather than using an average. The output is a single fixed-size d-vector representing the speaker's voice. The GE2E loss differs substantially from a binary classifier. Instead of training on pairs of recordings, each batch contains N speakers with M recordings each, and the loss trains all of them simultaneously, pulling each recording's embedding closer to its own speaker centroid and pushing it away from all others. One issue I encountered during implementation was the leave-one-out centroid. The GE2E loss requires that when measuring how close utterance X is to its own speaker centroid, utterance X must be excluded from that centroid calculation. Without this, the model can exploit the loss by making all embeddings for a speaker identical. My future work involves training the model on noisy data and adding VAD for live microphone input.

5.3 Leland

Completed Milestones

Create GitHub Project Structure: The repository structure for the project was created on GitHub, including the initial folder organization and version control setup, issues, roles, and timelines. This provides a foundation for managing source code, datasets, and documentation throughout the project.

Timeline Development: A project timeline was created outlining the major milestones and development stages. This timeline guides the overall progress of the project and helps ensure tasks are completed within the required timeframe; it is maintained within the GitHub project board.

End-to-End Data Extraction Pipeline: An automated script was implemented to perform end-to-end data extraction. The target dataset was selected and downloaded, and the pipeline processes the raw input data into a form suitable for further analysis and model training.

Training and Test Data Split: The extracted dataset was divided into separate training and testing sets. This step is essential for evaluating model performance and ensuring that the trained model generalizes well to unseen data.

Current Development Status: The project is currently moving into the model training and evaluation stage. The following tasks are in progress:

Model Checkpointing and Configuration Management: Work is underway to implement a system for saving trained model checkpoints and configuration files. This will allow models to be restored, reproduced, and compared across different training runs.

Performance Evaluation Across Noise Conditions: The project will evaluate model performance under different noise environments to test the robustness of the voice detection system. This includes testing with varying levels of background noise and analyzing how model accuracy changes under these conditions.

Summary

Milestones related to project design and data preparation have been successfully completed. The project now has a structured repository, a defined development timeline, a functioning data extraction pipeline, and properly prepared training and testing datasets. Current work is focused on model training, checkpoint management, and noise-condition evaluation.

5.4 Yilun

My responsibility in the project so far has focused on preprocessing raw audio into model-ready data, extracting frequency-domain features, defining evaluation procedures, and training a clean-data baseline model.

For the data extraction stage, I completed PI1 and PI2 by implementing preprocessing scripts to convert raw `.wav` recordings into structured features suitable for downstream modelling. In particular, the script standardizes audio into a consistent format, applies basic normalization and noise reduction, detects onset of the signal, and extracts the start frequency along with RMS, zero-crossing rate, duration, and normalized FFT bins. I then completed the FFT/DFT processing by implementing a script that converts time-domain audio into frequency-domain representations. The script summarizes important properties such as dominant frequency, dominant strength, spectral centroid, bandwidth, and rolloff. These outputs provide a clean feature layer for later analysis.

For the evaluation stage, I completed PI3 by creating a reusable verification-metrics script that computes false acceptance rate (FAR), false rejection rate (FRR), equal error rate (EER), ROC curve points, and AUC from model scores, with outputs saved in reproducible JSON and CSV formats. In addition, I implemented a feature-separability script that compares same-speaker and different-speaker pairs using cosine similarity and reports summaries such as similarity means, AUC, and EER. Together, these tools provide a consistent way to compare performance across different models, feature sets, and future experiments with noise conditions.

For the most recent model-training stage, I completed PI4 by building a baseline training pipeline using Mozilla Common Voice features. The current baseline filters out speakers without enough recordings, standardizes the re-

maining features, and trains a logistic regression model on clean audio data. On the filtered dataset, the baseline used 3,390 utterances from 152 speakers with 44 input features and achieved a mean validation accuracy of approximately 91.8% across repeated train-validation splits, with a top-5 accuracy of about 98.8%. Although this is still a baseline classification model rather than the final verification system, it shows that the extracted feature set carries useful information under clean conditions. This also provides a benchmark for later comparisons with more difficult conditions.

In summary, I have completed the majority of PI1 to PI4. For the remainder of the class, I will extend this work by comparing performance under noisy conditions, applying the evaluation scripts to later verification experiments, and contributing to the final ISMIR-style report.

6. PROJECT GOALS AND FUTURE WORK

6.1 Basic Goals

The basic goal of the project is to implement a functional prototype of a voice authentication system. At this level, the system imports an audio file of speech, extracts basic acoustic features such as MFCCs, and compares the extracted features with stored user data to determine whether the speaker matches the enrolled user.

6.2 Expected Goals

The expected goal is to build a noise-robust voice authentication pipeline that includes a microphone interface and a system for enrolling a user for authentication. This involves implementing voice activity detection (VAD) to isolate speech segments and evaluating authentication performance using training and testing datasets. The system incorporates a text-dependent prompt mechanism, where users speak a randomly generated word or phrase during authentication to reduce the likelihood of simple impersonation attempts. The expected outcome is a system demonstrating reliable authentication performance with accuracy, false acceptance rates, and false rejection rates reported.

6.3 Advanced Goals

The advanced goal is to further improve robustness by incorporating stronger speaker verification methods such as neural speaker embeddings (e.g., x-vector representations) and conducting extensive noise-robustness evaluation. The system will be tested under challenging acoustic conditions including very low SNRs and different types of background noise. The advanced stage will also include a real-time demonstration interface capable of performing live authentication. Additional security-related features, such as protection against replay attacks or enhanced noise-suppression techniques, may also be explored.

6.4 Future Work

Several directions remain for extending and improving the current system. The robustness of the authentication pipeline could be enhanced through integration of advanced neural speaker embedding models, such as x-vector or ECAPA-TDNN architectures, which have shown strong performance in recent speaker verification research. Future work may also expand the evaluation framework to include more realistic deployment scenarios, including testing with additional datasets containing reverberation, far-field microphones, and diverse recording conditions. Finally, the system could be extended to include security-focused features such as replay-attack detection, liveness verification, and continuous authentication during device usage.

7. REFERENCES

[1] J. Li, L. Deng, Y. Gong, and R. Haeb-Umbach, "An overview of noise-robust automatic speech recognition," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 22, no. 4, pp. 745–777, Apr. 2014.

[2] L. Zhang et al., "Voiceprint unlocking based on MFCC—exploration of voiceprint models different from fingerprint," in *Proc. 2024 IEEE 2nd Int. Conf. on Image Processing and Computer Applications (ICIPCA)*, Shenyang, China, 2024, pp. 763–769.

[3] Z. Meng, M. U. B. Altaf, and B.-H. Juang, "Active voice authentication," *Digital Signal Processing*, vol. 101, p. 102672, Jun. 2020.

[4] M. A. Alghamdi et al., "AudioUnlock: Device-to-device authentication via acoustic fingerprints," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 9, no. 3, Oct. 2025.

[5] H.-S. Bae, H.-J. Lee, and S.-G. Lee, "Voice recognition based on adaptive MFCC and deep learning," in *Proc. 2016 IEEE 11th Conf. on Industrial Electronics and Applications (ICIEA)*, Hefei, China, Jun. 2016, pp. 1542–1546.

[6] S. Das, J. Yang, and J. H. L. Hansen, "Quality measures for speaker verification with short utterances," *Digital Signal Processing*, vol. 88, pp. 66–79, May 2019.

[7] J. Ball, "Voice activity detection (VAD) in noisy environments," *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2312.05815

[8] M. Marras, "Speaker recognition in noisy environments," project page, 2019. [Online]. Available: https://mirkomarras.github.io/dl-voice-noise/

[9] B. H. Juang and S. Furui, "Automatic recognition and understanding of spoken language—a first step toward natural human-machine communication," *Proc. IEEE*, vol. 88, no. 8, pp. 1142–1165, Aug. 2000.

[10] S. Sun, B. Zhang, L. Xie, and Y. Zhang, "An unsupervised deep domain adaptation approach for robust speech recognition," *Neurocomputing*, vol. 257, pp. 79–87, Sept. 2017.

[11] L. Wan, Q. Wang, A. Papir, and I. L. Moreno, "Generalized end-to-end loss for speaker verification," *arXiv*, Oct. 2017. [Online]. Available: https://arxiv.org/abs/1710.10467

[12] J.-H. Kim, J. Heo, H.-J. Shim, and H.-J. Yu, "Extended U-Net for speaker verification in noisy environments," in *Proc. Interspeech*, 2022, pp. 590–594.

[13] Mozilla Foundation, "Mozilla Common Voice: An open multilingual speech corpus for machine learning," 2020. [Online]. Available: https://commonvoice.mozilla.org. Accessed: Feb. 3, 2026.

[14] D. Snyder, G. Chen, and D. Povey, "MUSAN: A music, speech, and noise corpus," *arXiv preprint arXiv:1510.08484*, 2015.

[15] C. Richey et al., "Voices obscured in complex environmental settings (VOiCES) corpus," in *Proc. Interspeech*, 2018, pp. 1566–1570.

[16] L. Zhang, S. Tan, J. Yang, and Y. Chen, "VoiceLive: A phoneme localization based liveness detection for voice authentication on smartphones," in *Proc. 2016 ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, Vienna, Austria, 2016, pp. 1080–1091.

Appendix A: Normalization Diagnostic Figures

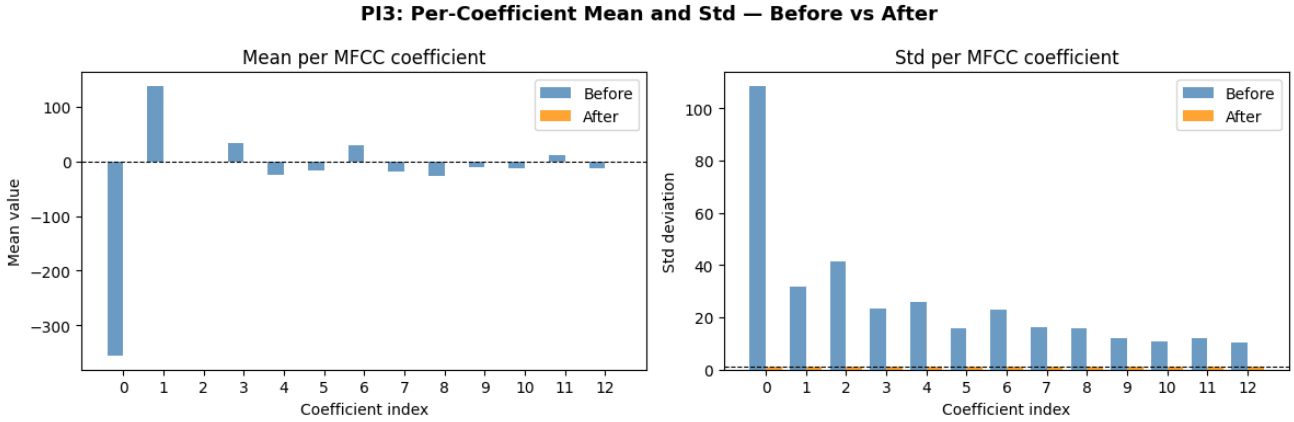

Figure 1. Per-coefficient mean and standard deviation before vs. after normalization. Raw MFCC (blue) showed large, inconsistent means and standard deviations across coefficients. After CMVN normalization (orange), all 13 coefficients collapse to approximately zero mean and unit variance, confirming that the normalization step successfully removed per-coefficient bias and scale differences.

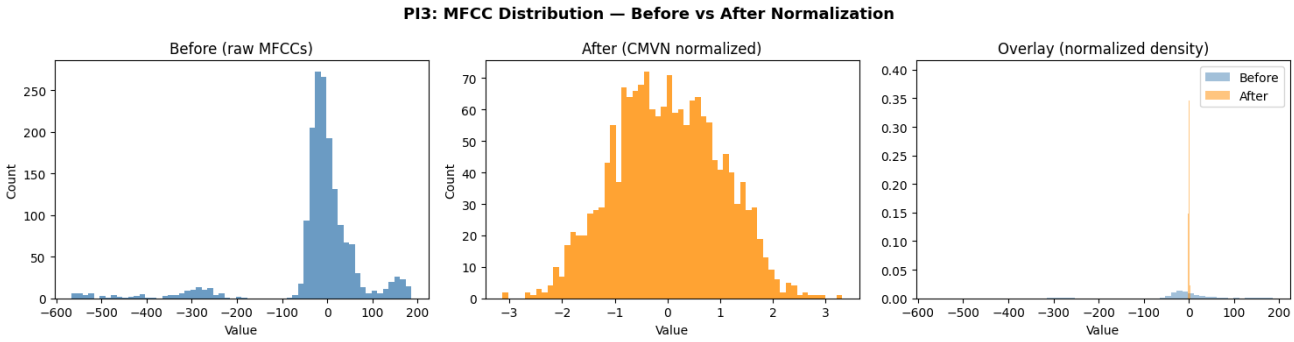

Figure 2. MFCC value distribution before vs. after normalization. The raw MFCC distribution (left) is heavily concentrated near zero with a long tail extending to -600, reflecting the uneven scale of raw features. After CMVN (center), the distribution becomes an approximate Gaussian spread, centered at zero and within the range $[-3, 3]$.

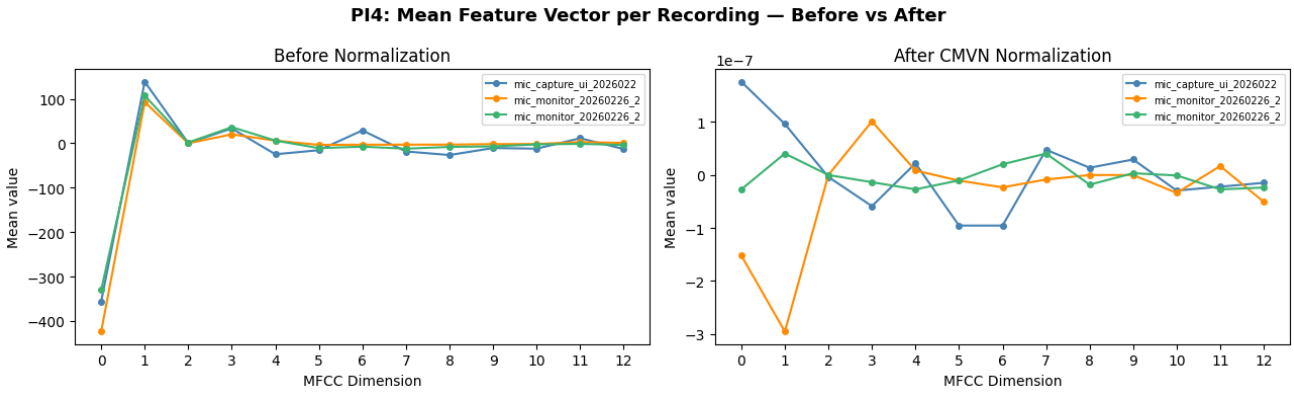

Figure 3. Mean feature vector per recording before vs. after normalization. Before normalization (left), the three recordings diverge significantly at lower coefficients. After CMVN (right), all three converge to near-zero across all dimensions, with differences on the order of 10^{-7}.

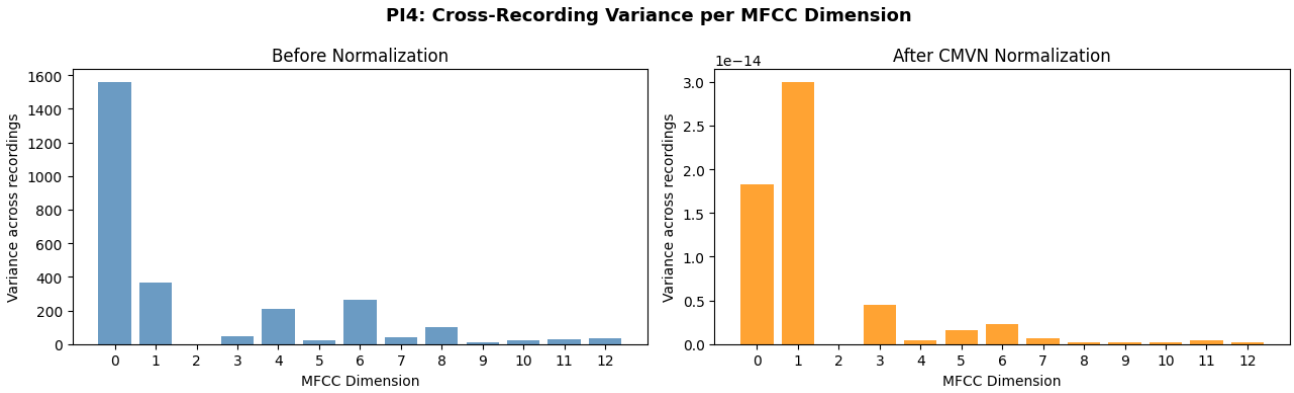

Figure 4. Cross-recording variance per MFCC dimension before vs. after normalization. Before normalization (left), variance at coefficient zero exceeds $1{,}500$, but after normalization (right) the entire plot becomes scalable to 10^{-14}. This confirms the pipeline produces consistent model inputs across different recording conditions.